

Écosystème Nkentseu — description générale

Vue d'ensemble (présentation). Pour la vision détaillée + la feuille de route priorisée, voir [ECOSYSTEM.md](#).

La thèse

Rihen construit un **écosystème logiciel intégré verticalement et possédé de bout en bout** — *du métal jusqu'à l'IDE et l'IA* — chaque couche écrite **from scratch**, sans dépendance tierce, **zéro-STL**, mémoire via **NKMemory**, conventions **NkPascalCase**.

Si tu possèdes chaque couche, tu n'es bloqué par personne.

Le moteur Nkentseu (couches, acycliques)

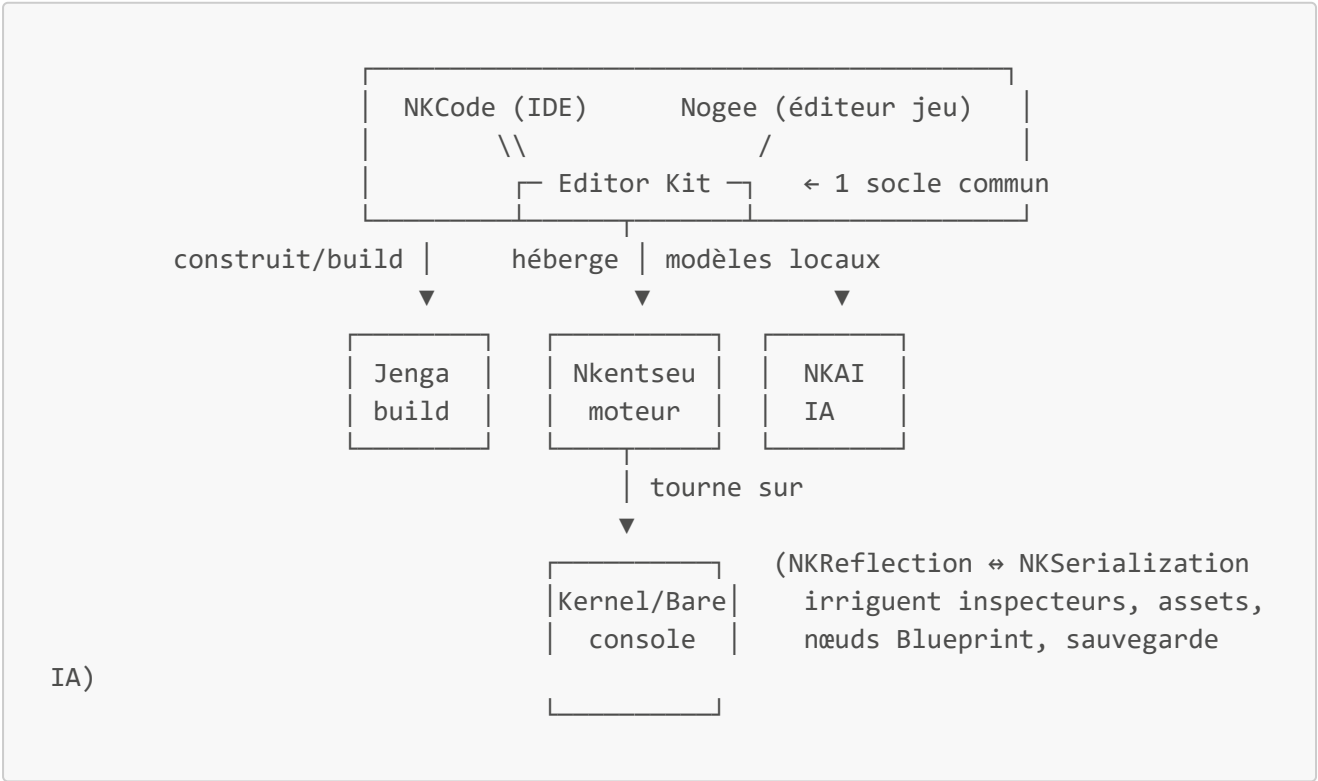
Couche	Modules	Rôle
Foundation	NKPlatform , NKCore , NKMemory , NKContainers , NKMath	détection OS/CPU, types/macros, allocateurs & smart pointers, conteneurs zéro-STL, maths
System	NKLogger , NKFileSystem , NKStream , NKThreading , NKTime , NKNetwork , NKReflection , NKSerialization	services bas niveau ; réflexion + sérialisation = la clé de voûte
Runtime	NKWindow (13 backends), NKEvent , NKImage , NKFont , NKCanvas (2D), NKRHI (3D, 6 backends) + NKSL (cross-compileur de shaders maison), NKRenderer (3D sur RHI), NKAudio , NKCamera , NKUI (UI immédiat actuel) → NKGui (réécriture en cours)	fenêtrage, rendu 2D/3D, audio, UI
Engine	Noge	moteur de jeu 2D/3D (ECS, scènes, gameplay) sur NKRenderer

La famille de produits (3 éditeurs, 1 socle)

Produit	Rôle	Analogue	État
---------	------	----------	------

Produit	Rôle	Analogue	État
Jenga	Système de build (DSL Python cross-plateforme, sans CMake) : compile / package / déploie <i>tout</i>	—	☑ v2.0.7 publiée
Editor Kit (NKEditorKit)	Socle d'éditeur partagé : shell dockable, panneaux, inspecteur, node-graph, palette de commandes, thème — construit une fois	—	🌀 en cours
NKCode	IDE / éditeur de code : texte + langages visuels (Blueprint/Blocks) + extensions + agents IA . App mince sur l'Editor Kit	≈ VSCode / Rider	🏗️ scaffold
Nogee	Éditeur de jeu : scènes / niveaux / assets / inspecteur, sur Noge. App mince sur l'Editor Kit	≈ Unity Editor / UnrealEd	🏗️ à démarrer
NKAI (Kernel/AI)	Framework IA from scratch : tenseurs / NN / RL / LLM + agents + sim de civilisation ; modèles locaux pour NKCode	—	🏗️ scaffold (pause)
Kernel/Bare	OS bare-metal d'une console maison faisant tourner Nkentseu	—	🏗️ scaffold (pause)
Jeux-preuves	Pong , Nkoug (plateforme 2D), Mou (éveil maternelle), Songoo ... valident le moteur	—	⏸️ en pause

Comment tout s'imbrique



- **Jenga construit tout · Nkentseu rend/anime tout.**
- **NKCode** et **Nogee** partagent ~70 % de plomberie → **un seul Editor Kit** (chacun devient une app mince dessus).
- **NKReflection** ↔ **NKSerialization** = tissu transverse (inspecteurs, assets, nœuds, sauvegardes) → **5+ consommateurs**, donc le **goulot prioritaire**.
- **NKAI** donnera des **agents 100 % locaux** à NKCode ; **Kernel/Bare** fait tourner Nkentseu sur la **console maison**.
- Le **scripting visuel** (Blueprint/Blocks) vit dans l'Editor Kit (substrat Graph commun), exposé à Nogee (gameplay) **et** à NKCode.

Plateformes

- **Production** : Windows, Linux (xlib / xcb / wayland), Android, Web/WASM.
- **Prêtes** : macOS, iOS, HarmonyOS.
- **Partiel** : Xbox (GDK).
- **Licence requise** : Switch, PS4/5.

Principes transverses

- **Zéro-STL** ; allocation **exclusivement via NKMemory** (jamais **new/delete**).
- **Découpage en couches strict et acyclique** (Foundation → System → Runtime → Engine → App).
- **RHI + cross-compileur de shaders maison** (NkSL → SPIR-V / GLSL / HLSL / MSL).
- Nomenclature **NkPascalCase** ; build piloté par **Jenga** (toolchains natives, pas de CMake/Makefile).

Le cap actuel

Roadmap officielle (anti-dispersion) : **1)** finir **NKRenderer** (saga shaders NkSL → HLSL) · **2)** mûrir **NKReflection** ↔ **NKSerialization** · **3)** **Editor Kit** (construit une fois) · **4)** apps minces **NKCode / Nogee**. Devise : *une tranche qui marche > un scaffold de plus.*

Chantier en cours (transverse UI) : réécriture de NKUI en NKGui — un framework UI neuf (immédiat **et** retenu, qualité ImGui), parce que l'UI sous-tend **tous** les éditeurs. Déjà fait : l'étalon **ImGuiRef** (vrai Dear ImGui qui tourne sur Nkentseu), puis NKGui Phases 1–2 (architecture + cœur : draw list, machine d'interaction, z-ordre correct, boutons à rafale). Phase 3 en cours (**texte via NKFont**). À terme, **NKGui remplace NKUI sous l'Editor Kit**.